Development of an Optical Tracking System for Real-time Robotic Applications

Shannon Pitman

Author: Shannon Pitman
Supervisor: Arnold Pretorius
Date: September 2025

MechatronicsSystems.Group
Department of Mechanical Engineering
University of Cape Town

Submitted in partial fulfilment of the requirements for the degree of
°

Abstract

Engineering students often need a clear, practical model for writing dissertations; this document provides an overview tailored to final-year mechatronics projects and demonstrates effective use of a LaTeX thesis template. Many student reports lack consistent structure, rigorous scoping, and appropriate use of figures, tables, code listings, and citations, which impedes readability and assessment.

This work contributes an exemplar that explains what each report section should contain, offers typical subheadings, and embeds working examples of template elements to copy and adapt. The document is implemented in XeLaTeX using the `memoir` class with chapters for Introduction, Literature Review, Methodology, Results, Discussion, Conclusions, Recommendations, and Appendices, along with acronyms and bibliography support; the result is a shareable guide that doubles as a starting point for students to produce professional, evidence-based engineering reports.

How to write an abstract in 5 minutes?

Answer the following questions in narrative form with no more than two sentences per question:

- *What is the background to the project?*
- *What is the problem that is being solved?*
- *What is the contribution of the project to this problem?*
- *What is the main conclusion of the project?*

Contents

List of Figures iii

List of Tables iv

Acronyms v

1 Introduction 1
- 1.1 Background and Motivation . . . 1
- 1.2 Context and Related Work . . . 1
- 1.3 Aims, Objectives and Contributions . . . 1
- 1.4 Project Scope and Limitations . . . 1
- 1.5 Report Outline . . . 1

2 Literature Review 2
- 2.1 Introduction . . . 2
- 2.2 Overview of Motion Capture . . . 2
- 2.3 Survey of Optical Motion Capture . . . 2
- 2.4 Applications of Optical Motion Capture . . . 2
- 2.5 Limitations of Existing Approaches . . . 2
- 2.6 Summary . . . 2

3 System Specifications 3
- 3.1 Introduction . . . 3
- 3.2 Overall System Architecture . . . 3
- 3.3 OCP Subsystem Requirements . . . 3
  - 3.3.1 Performance Requirements . . . 3
  - 3.3.2 Functional Requirements . . . 3
- 3.4 OMC Subsystem Requirements . . . 3
  - 3.4.1 Performance Requirements . . . 3
  - 3.4.2 Functional Requirements . . . 3
- 3.5 Constraints and Assumptions . . . 3

4 Optical Motion Capture 4
- 4.1 Literature Review . . . 4
  - 4.1.1 Estimation Algorithms . . . 4
  - 4.1.2 Hardware Considerations for Camera-Based Sensing . . . 4
- 4.2 Camera Selection and System Integration . . . 4

4.3 Theory Development 4
4.3.1 Rigid-body Motion 4
4.3.2 Image Acquisition and Formation 4
4.3.3 Camera Calibration 5
4.3.4 Image Processing 5
4.3.5 Triangulation 5
4.3.6 State Estimation 5
4.4 Tracking Algorithm Design 5
4.4.1 Software Architecture 5
4.5 Implementation 5
4.5.1 Calibration Procedure 5
4.5.2 Graphical User Interface 5
4.6 Experimental Testing Setup 5
4.6.1 Physical Configuration 5
4.6.2 Reference System and Ground Truth 5
4.6.3 Test Trajectories 5
4.6.4 Data Acquisition and Processing Pipeline 5
4.7 Results and Discussion 6
4.7.1 Calibration Accuracy 6
4.7.2 Static Localisation Performance 6
4.7.3 Dynamic Tracking Performance 6
4.7.4 Filter Comparison 6
4.7.5 Real-time Performance 6
4.7.6 System Limitations and Failure Analysis 6

5 Optimal Camera Placement 7
5.1 Literature Review 8
5.1.1 Background of OCP 8
5.1.2 Motivation for Optimal Placement 9
5.1.3 Optimisation Approaches for Sensor Placement 10
5.1.4 Modelling the Problem 13
5.1.5 Summary 18
5.2 Theoretical Framework 18
5.2.1 Genetic Algorithm Terminology 18
5.2.2 Encoding Schemes 19
5.2.3 Selection Methods 20
5.2.4 Crossover Techniques 20
5.2.5 Convergence Analysis 20
5.3 Methodology 20
5.4 Methodology 20
5.4.1 Algorithm Configuration 20
5.4.2 Cost Function 27
5.4.3 Termination Criteria 31
5.5 Experimental Verification Framework 31
5.5.1 Verification Objectives 32
5.5.2 Ground Truth and Reference Metrics 32
5.5.3 Experimental Protocol 33

5.5.4 Linking Simulation to Measurement 33
5.6 Results and Discussion 34
5.6.1 Convergence Behaviour 34
5.6.2 Cost Function Evaluation 34
5.6.3 Parameter Sensitivity Analysis 34
5.6.4 Optimal Placement Configurations 34
5.6.5 System Limitations 34

6 Conclusion and Future Work 35
6.1 Summary of Contributions 35
6.2 Key Findings 35
6.3 Limitations 35
6.4 Recommendations for Future Work 35

References 36

List of Figures

5.1 Vertex tricolouring of a 13-vertex polygon. 9
5.2 Distribution of heuristic-based approaches used for OCP in the reviewed literature 12
5.3 Flowchart of the guided genetic algorithm. 21

List of Tables

5.1 Classification of the reviewed Optimal Camera Placement (OCP) literature by solver, placement space, and optimisation objective. 16
5.2 Comparison of biological and genetic algorithm terminology. 19
5.3 Genetic algorithm parameter settings. 23

Acronyms

AGP Art Gallery Problem 8–10, 14, 18

BIP Binary Integer Program 10–13, 17, 22

DNA deoxyribonucleic acid 19

FOV Field of View 8

GA Genetic Algorithm 10, 11, 13

GDOP Geometric Dilution of Precision 11, 12, 14, 17

MILP Mixed-Integer Linear Program 10, 11, 22

MoCap Motion Capture 8, 9, 11, 12, 14, 15, 17, 20

NP-hard Non-deterministic Polynomial-time hard 7

OCP Optimal Camera Placement vi, 7–11, 13–17, 20, 23

PSO Particle Swarm Optimisation 7, 10, 11, 13

Introduction 1

1.1 Background and Motivation

1.2 Context and Related Work

1.3 Aims, Objectives and Contributions

1.4 Project Scope and Limitations

1.5 Report Outline

Literature Review 2

2.1 Introduction

2.2 Overview of Motion Capture

2.3 Survey of Optical Motion Capture

2.4 Applications of Optical Motion Capture

2.5 Limitations of Existing Approaches

2.6 Summary

System Specifications 3

3.1 Introduction

3.2 Overall System Architecture

Block Diagram

3.3 OCP Subsystem Requirements

3.3.1 Performance Requirements

3.3.2 Functional Requirements

3.4 OMC Subsystem Requirements

3.4.1 Performance Requirements

3.4.2 Functional Requirements

3.5 Constraints and Assumptions

Optical Motion Capture 4

4.1 Literature Review

4.1.1 Estimation Algorithms

4.1.2 Hardware Considerations for Camera-Based Sensing

4.1.2.1 Camera Capabilities

Resolution, latency, iframe rate, microcontroller...

4.1.2.2 Communication Architectures

Wired vs wireless: PoE, Wi-Fi, Bluetooth Synchronous vs Asynchronous

4.2 Camera Selection and System Integration

4.3 Theory Development

4.3.1 Rigid-body Motion

4.3.1.1 Reference Frames

4.3.1.2 Representing Pose

4.3.2 Image Acquisition and Formation

Computer vision, light and colour, image formation

4.3.2.1 Colour Spaces

4.3.2.2 Camera Model and Projection

4.3.3 Camera Calibration

4.3.4 Image Processing

4.3.4.1 Marker Design and Detection

4.3.5 Triangulation

4.3.6 State Estimation

4.3.6.1 The Kalman Filter

4.3.6.2 The Extended Kalman Filter

4.3.6.3 The Invariant Extended Kalman Filter

4.3.6.4 The Two-Frames Group In-EKF

4.4 Tracking Algorithm Design

4.4.1 Software Architecture

4.5 Implementation

4.5.1 Calibration Procedure

4.5.2 Graphical User Interface

4.6 Experimental Testing Setup

4.6.1 Physical Configuration

4.6.2 Reference System and Ground Truth

4.6.2.1 Spatial and Temporal Alignment

How the system's coordinate frame was aligned with OptiTRack - will camera be in their optimised position versus OptiTrack's ad hoc arrangement? How the two data streams would be temporarily aligned

4.6.3 Test Trajectories

4.6.4 Data Acquisition and Processing Pipeline

flow chart

4.7 Results and Discussion

4.7.1 Calibration Accuracy

4.7.2 Static Localisation Performance

4.7.3 Dynamic Tracking Performance

4.7.4 Filter Comparison

4.7.5 Real-time Performance

4.7.6 System Limitations and Failure Analysis

Optimal Camera Placement 5

The quality of a multi-camera localisation system is determined as much by the geometric arrangement as by hardware. Sensor noise, calibration accuracy, and algorithmic performance all matter; yet even a perfectly calibrated, high-quality camera system running a sophisticated algorithm can still fail to localise the target if its cameras are arranged poorly. Determining the OCP requires both a model of how error arises and subsequently propagates, and an effective optimsation formulation that is capable of navigating a large search space timeously.

When the sensor placement is suboptimal, the inaccuracy tends to compound in two ways. The first results from the geometric measurement uncertainty. Cameras discretise their worldview into pixels; the target is projected onto a finite grid of pixels, and its position is inferred from this quantised image (as explained in **??**). The further a target is from a camera, the fewer pixels it subtends, reducing the spatial resolution and increasing the resulting position estimate's variance. Moreover, when two cameras observe a target from nearly parallel viewing angles, small pixel-level uncertainties preclude triangulation [1]. The resulting depth estimate becomes ill-conditioned, degrading the tracking prediction. Hence, in these avoidable configurations, the limitation of the tracking system is not the sensor itself but the geometry.

The second degrading source is visibility loss. Intuitively a camera cannot measure what it cannot see. Occlusion, whether caused by invariant static obstacles in the environment or dynamically occurring by the target itself reduce the number of measurements [2]. Similarly, limited coverage may result in area's being unobserved. Therefore, it may be necessary to optimise for an arrangement that rewards redundant views whilst still ensuring that most of the flight space is observable [3]. Thus, the arrangement of the sensors can exacerbate localisation error beyond the intrinsic noise characteristics of the cameras themselves.

Optimising placement, however, is combinatorially explosive, and classified as Non-deterministic Polynomial-time hard (NP-hard) [4]. As the number of inputs (camera, candidate placements, and workspace size) grow, the number of possible configurations grows exponentially; consequently, an exhaustive search becomes intractable. In this context, NP-hard denotes that the problem becomes increasingly computationally challenging to find a guaranteed global solution with a multitude of inputs. The literature suggests that heuristic- or metaheauristic-based algorithms, such as genetic algorithms, greedy methods, or Particle Swarm Optimisation (PSO) approaches, are adopted because of their proficiency in exploring large, unsmooth, multi-objective search spaces efficiently [Citation needed]. Alternatives, like integer

and mixed-integer programming attempt to find globally optimal solutions but under more heavily imposed constraints. Therefore, each optimisation technique reflects a tradeoff between optimality guarantees, scalability, efficiency and modelling fidelity.

5.1 Literature Review

Optimising camera placement is not optional; it should be a prerequisite for accurate tracking. This section reviews the literature on the OCP problem. First, the optimisation strategies employed to search the ill-defined, non-convex search space in sufficiently good time are examined. Second, an examination of how the sources of measurement uncertainty, quantisation, triangulation geometry, and occlusion, have been characterized and modelled is provided. Together, this provides the state of the art in designing geometrically valid and computationally reasonable multi-camera tracking systems.

5.1.1 Background of OCP

OCP descends from the well-studied Art Gallery Problem (AGP), which established the combinatorial foundations of visibility-based sensor placement. The AGP minimises the number of omnidirectional vertex guards required to observe an entire 2D polygon with n vertices. Fisk proved, via triangulation and 3-colouring, that $\lfloor \frac{n}{3} \rfloor$ guards are always sufficient for covering any simple polygon, as demonstrated in Figure **??**

[Review O'Rourke (1987) Art Gallery Theorems and Algorithms AGP complexity is NP-hard, OCP is also NP-hard]. However, the AGP's usefulness is bounded by its assumptions: where guard are endowed with unlimited sensing range for 360 ° Field of View (FOV), and perfect visibility. Real cameras violate these assumptions; where physical sensors have a limited range of marker detection, resolution that degrades with distance, a finite FOV, and the target may be occluded during observation.

The AGP is bounded by its approach to solely guarantee that all the points in the workspace are seen by idealised observers. Whereas, the OCP objective is to determine the most accurate camera configuration to accurately reconstruct the points. This shift highlights how the OCP frames the problem as one to ensure quality of reconstruction rather than feasibility. In Motion Capture (MoCap), a marker that is only visible by one camera cannot be triangulated; Olague and Mohr emphasise how a marker visible by two (anti)parallel cameras can be triangulated but with large uncertainty in the depth direction; and a marker seen by two or more cameras with sufficient angular separation yields a well-conditioned reconstruction [5]. Therefore, although the OCP inherits the NP-hard structure of the AGP it is necessary for the optimisation to encode that every point of interest is visible by a sufficient number of cameras in a geometrically favourable arrangement rather than described as a binary coverage test.

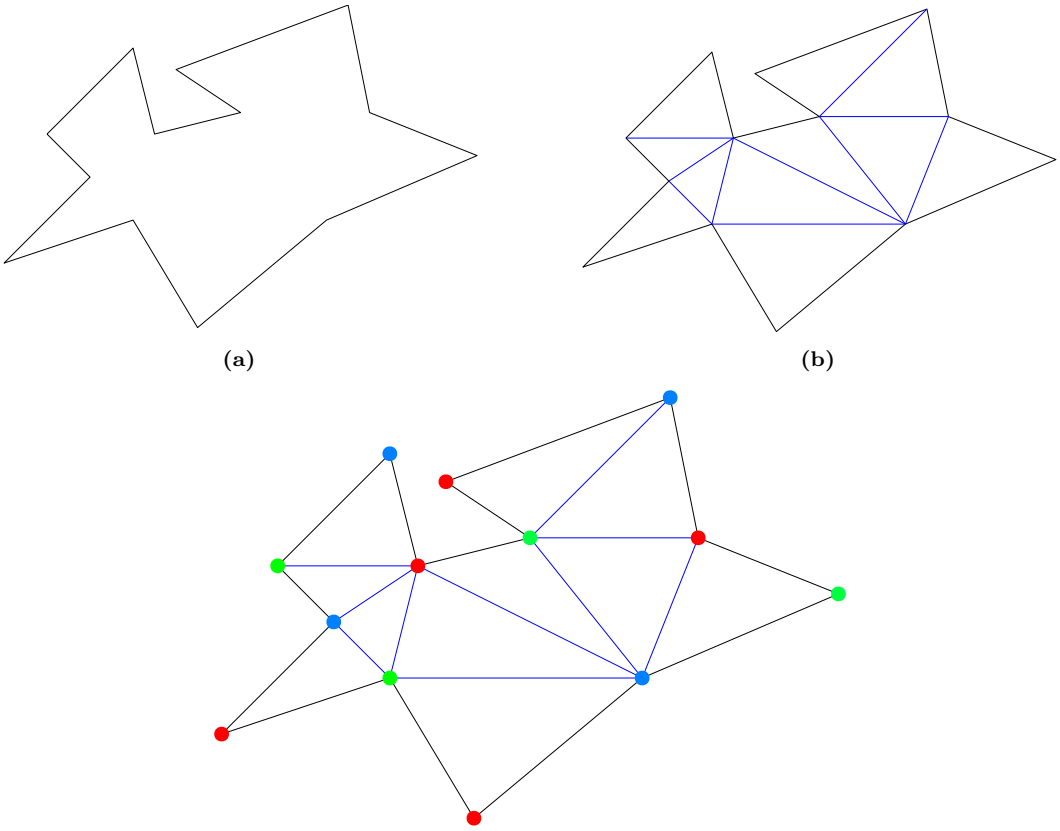


Figure 5.1: In this example, a polygon with $n = 13$ vertices is first triangulated (b), then each vertex is assigned one of three colours such that no triangle shares two vertices of the same colour (c); placing guards at all vertices of the least frequent colour (green or blue) guarantees complete coverage.

5.1.2 Motivation for Optimal Placement

The need for OCP emerges across diverse applications, each imposing different constraints on what embodies optimal placement. Surveillance and monitoring systems are most closely connected to the AGP formulation, where spatial coverage is maximised to ensure that no region goes unobserved, tolerating lower reconstruction accuracy in exchange for exhaustiveness [6]. By contrast, MoCap systems require each marker in the scene to be visible to at least two cameras, with sufficent angular separation simultaneously for accurate triangulation [3, 7, 8]. Autonomous driving scenarios necessitate a full 360° surround view, stitching adjacent camera frustrums to provide the driver and autonomous system with reliable proximity warnings of

neighbouring obstacles [9]. Augmented and virtual reality applications face more severe mounting constraints and an abundance of unpredictable dynamic occlusion, as the human user moves freely within the encapsulating simulation equipment [2]. Despite this diversity, all camera placement optimisers in the literature share the common challenge: the quality of the system's output (surveillance or surrounding map, reconstructed trajectory, or a pose estimate) hinges on where the cameras are placed.

Without a formal optimiser, camera networks are typicaaly set-up through ad-hoc designer intuition or rule-of-thumb guidelines, for e.g. distribute the cameras evenly around the perimeter. While this may appear as an efficient approach, the designer has no means of assessing whether a different arrangement would reduce localisation error. The resulting configuration is neither verifiably optimal nor is the theoretical performance quantified. This is particularly pertinent when the number of cameras is constrained by budget, as each camera should be placed to contribute advantageously to the overall network. Rahimian et al. demonstrated that the performance cost of forgoing optimisation can be substantial. Their PSO OCP algorithm enabled their OptiTrack system to reconstruct all 37 markers on 86% of captured frames, compared to only 30% under an ad-hoc configuration [2]. This nearly tripled improvement achieved by repositioning the cameras in the same room illustrates the performance gap between optimised and unguided placements.

Thus, the motivation to optimise is evident but the problem itself is involved. The AGP, from which the OCP descends, is NP-hard even under its idealised two-dimensional, panoptic-guard assumptions [4]. Subsequently, the OCP is even more complex since it requires more inputs: each camera is described both by a three-dimensional position (x, y, z) and orientation (θ, ϕ, ψ), and the sensor has constraints, such as a finite field of view and depth of field that the AGP does not consider [Citation NEEDED/ section reference here about AGP being NP hard and thus with OCP having more constraints it becomes increasingly challenging]. The placement of k cameras over a subsampled three-dimensional workspace creates a search space that grows combinatorially. If the mounting locations are discretised, further reducing the cardinality of the problem, into N candidate positions, each allowing M candidate orientations, then the number of distinct camera configurations is $\binom{NM}{k}$. For a modest $N = 50$ possible positions, $M = 8$ orientations, and $k = 6$ cameras this exceeds 5^{12} combinations; every one of which requires its own assessment based on the cost function which involves visibility quality and or triangulation geometry computations. Consequently, most investigators resort to meta-heuristic-based approximation algorithms which, albeit without global optimality guarantees, offer the only practical path to obtain acceptable solutions within reasonable computation time.

5.1.3 Optimisation Approaches for Sensor Placement

The OCP literature is conventionally divided into two groups of solver: mathematical programming (typically Binary Integer Program (BIP) or Mixed-Integer Linear Program (MILP)) and metaheuristic search (Genetic Algorithm (GA), PSO, ??).

This framing is not wrong, but it obscures a more consequential distinction. The choice that most constrains both the solver and the fidelity of the resulting solution is whether the placement space is discretised. In a discrete formulation, the continuous room is sampled onto a finite grid of candidate positions and orientations before optimisation begins; the solver then selects a subset of these candidates. In a continuous formulation, the solver operates directly on real-valued camera parameters, position and orientation, without any prior commitment to a grid. Everything downstream, including which algorithms are admissible, what optimality guarantees are available, and how faithfully the cost function can represent the underlying physics, follows from this single modelling decision.

Discrete formulations are the natural domain of mathematical programming. BIP and MILP encode the placement problem as a selection over binary decision variables: each candidate sensor pose is either included or excluded. This structure admits exact solvers such as CPLEX, which exploit branch-and-bound to compute optimality bounds, and permits the designer to prove that a solution is within a known factor of the global optimum [4]. Kirchhof presented both a BIP and a MILP formulation for indoor sensor placement, using Geometric Dilution of Precision (GDOP) as the placement quality metric, and demonstrated that for moderately sized instances the solver can certify optimality. Dai formulated the multi-camera MoCap problem as a BIP with an occlusion culling factor, proving a logarithmic performance ratio $H(k)$ for the greedy sub-solver used to handle larger instances [11]. The cost, however, is modelling fidelity. BIP requires that the objective function, visibility checks, and accuracy metrics all be expressed as linear or quadratic functions of binary variables. In practice, this either limits the sensor model to what can be linearised, or it produces problem instances too large for exact solvers, forcing the use of approximate sub-solvers such as greedy heuristics or LP relaxations even within the mathematical programming framework [**zhaoApproximateTechniquesSolving2013**]. Discretisation also introduces a modelling artefact: the optimal solution is constrained to lie on the grid. The coarser the grid, the larger the potential gap between the discrete optimum and the true continuous one; yet finer grids produce BIP instances that grow combinatorially, which was the motivation for discretising in the first place.

Meta-heuristic algorithms approach the problem from the opposite direction. Rather than requiring the cost function to conform to an algebraic structure, they treat it as a black box: the solver submits a candidate camera configuration and receives a scalar cost, where the optimiser has no knowledge of or requirement on the function's internal form. GAs, PSO, and ?? all operate in this manner, exploring the search space through stochastic, population-based or neighbourhood-based mechanisms that require neither gradient information nor a linearised objective. This formulation-independence is precisely what permits complex cost functions involving ray-cast visibility checks, probabilistic dynamic occlusion models, or GDOP-based triangulation metrics, none of which can be expressed as linear constraints. As shown in Figure **??**, GAs are the most prevalent metaheuristic in the surveyed OCP literature, followed by PSO, with ?? and greedy approaches appearing less frequently. Note that the placement space itself may still be discretised under this paradigm:

Zhou et al., for instance, discretised candidate sensor positions at regular intervals along the workspace perimeter before applying a multi-objective metaheuristic [8]. The distinction between algebraic and black-box solvers is therefore orthogonal to the distinction between discrete and continuous placement spaces, and conflating the two obscures the actual design choices made by each author.

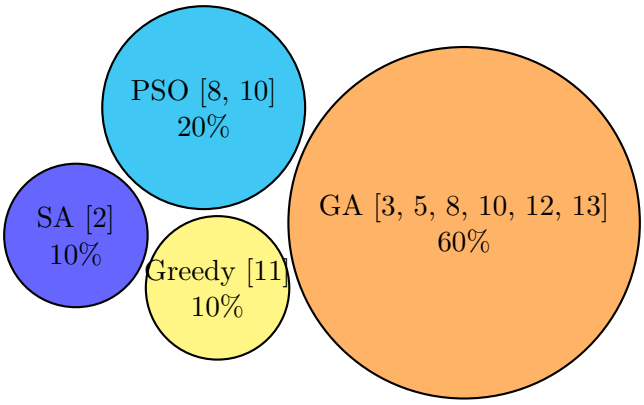


Figure 5.2: Distribution of heuristic-based approaches used for OCP in the reviewed literature

The diversity of solvers in the literature is not arbitrary; each reflects a deliberate trade-off between the guarantees the designer was willing to forgo and the modelling complexity they needed to retain. Kirchhof adopted BIP because indoor localisation with a small number of candidate poses permitted exact optimality certification, and his GDOP metric could be linearised without excessive distortion; but he also presented a continuous NLP alternative for comparison, acknowledging that the discrete formulation imposes a ceiling on solution quality [4]. Dai retained the BIP framework for a larger MoCap problem but conceded the need for a greedy sub-solver once the instance size exceeded what branch-and-bound could handle, proving only a logarithmic approximation ratio rather than exact optimality [11]. At the other extreme, Rahimian and Kearney chose simulated annealing because their occlusion-based error metric, which integrates visibility over all possible orientations of a dynamic occluder whilst simultaneously enforcing triangulability, cannot be decomposed into linear constraints; SA's probabilistic acceptance of worse solutions also provided a mechanism to escape the local minima that pervade such non-convex landscapes [2]. Olague and Mohr selected a real-valued GA for a similar reason: the covariance of 3D point reconstruction that they sought to minimise is a nonlinear function of continuous camera parameters, and the GA's population-based search offered broad exploration without requiring gradient information [5]. Aissaoui et al. adopted a GA but found that an unguided population converged too slowly over the high-dimensional search space of a multi-camera MoCap system; their response was to seed the initial population using a distribution-and-estimation technique that restricted the search to a physically plausible capture zone based on an expected trajectory, producing what they termed a "guided genetic algorithm" [3]. Ziegler et al. avoided meta-heuristics entirely, smoothing the binary visibility function with a tanh approximation to render the problem differentiable and solving the resulting nonlinear program with IPOPT, a gradient-based interior-point solver [7]. The pattern that emerges is instructive: as the cost function retains physical fidelity, the

solver must become more flexible, and the designer must accept weaker guarantees about the quality of the returned solution.

The most revealing trend in the recent literature is not the dominance of any single algorithm but the growing prevalence of hybrid methods. Zhou et al. combined PSO with a GA explicitly to compensate for the tendency of each to stagnate: PSO converges rapidly but is prone to premature convergence in multimodal landscapes, while the GA's crossover and mutation operators maintain diversity but converge slowly on their own [8]. Zhang et al. compared and combined GA and PSO formulations for 3D object coverage [10]. Aissaoui et al.'s guided GA is itself a bit of a hybrid: domain knowledge is used to reduce the effective search space before the stochastic optimiser begins, combining deterministic preprocessing with evolutionary search [3]. That investigators keep bolting supplementary mechanisms onto individual solvers is itself the most compelling evidence that no single algorithm has proven sufficient. The problem resists tidy solutions, and the literature has responded not by finding the right algorithm but by combining imperfect ones.

Returning to the discrete-versus-continuous axis: the literature's treatment of the placement space is seemingly inconsistent. Kirchhof sampled sensor poses at regular intervals along wall edges and mountable surfaces as a prerequisite for his BIP [4]. Dai discretised candidate locations into a grid of positions with associated orientation bins [11]. Zhou et al. placed candidate positions at regular perimeter intervals before applying their hybrid meta-heuristic [8]. In each case, discretisation was adopted to make the problem more computationally manageable, but none of these authors systematically examined what was lost in doing so. On the continuous side, Olague and Mohr encoded camera parameters as real-valued chromosomes [5]; Aissaoui et al. represented each individual as a real-valued vector of translation and orientation parameters [3]; Rahimian and Kearney optimised over continuous camera poses constrained to user-specified geometric regions [2]; and Ziegler et al. solved a continuous nonlinear program directly [7]. Kirchhof is uniquely instructive because he presented both a discrete BIP and a continuous NLP formulation and compared their runtime and solution quality, but this dual treatment remains the exception rather than the norm. What is absent from the literature is a single system that is designed to operate across the spectrum: fully continuous placement when the mounting environment permits it, and discrete placement when physical constraints such as fixed bracket locations demand it, with the same underlying cost function and genetic operators adapted to both modes. (add when the search and mounting space was 3d or constrined to 2d)

5.1.4 Modelling the Problem

maybe restructure the discrete vs contnuous aspect here

5.1.4.1 Optimisation Objectives

The most intuitive OCP objective, and historically the first, is spatial coverage: ensuring that every point in the workspace is observed by at least one camera, or by at least k cameras for redundancy. This objective descends directly from the

AGP, where success is defined as a binary visibility test, a point is either seen or it is not. Bodor et al. maximised an aggregate observability measure weighted by task-specific motion statistics, which is fundamentally a coverage metric, albeit one that prioritises regions of expected activity over uniform spatial exhaustiveness [6]. Zhou etal. included a spatial coverage term in their multi-objective function to ensure that the workspace is adequately monitored [8]. For surveillance applications, where the goal is exhaustive monitoring rather than precise reconstruction, coverage-dominant objectives are often sufficient. For MoCap, they are not. A marker that is visible to only one camera cannot be triangulated. Coverage guarantees observation; it does not guarantee measurement.

For applications requiring three-dimensional reconstruction, the objective must encode measurement quality, and the literature has characterised this quality through three distinct but complementary mechanisms. The first is triangulation geometry. When two cameras observe a target from nearly parallel viewing angles, the resulting depth estimate becomes ill-conditioned regardless of how precisely each camera localises the target in its own image plane. Kirchhof used GDOP explicitly as a metric of geometric quality, penalising configurations where the angular separation between sensor lines-of-sight is insufficient for well-conditioned triangulation [4]. Ziegler et al. constrained the convergence angle between optical axes and demonstrated that triangulation error increases exponentially as the convergence angle approaches 0° or 180° [7]. Olague and Mohr minimised the maximum diagonal element of the covariance matrix of 3D point reconstruction, capturing the worst-case error propagation for a given configuration [5]. The second mechanism is resolution uncertainty, which operates before triangulation at the image-formation stage. Wu, Sharma, and Huang derived analytical uncertainty bounds arising from pixel quantisation as a function of camera-target geometry, showing that the further a target lies from the camera and the more oblique its projection, the larger the variance in the resulting position estimate [1]. The third is dynamic occlusion. Rahimian and Kearney defined a metric that integrates target visibility over all possible orientations of a vertical planar occluder, simultaneously enforcing that the visible camera pairs satisfy a triangulability constraint on their convergence angle [2]. Dai modelled occlusion probabilistically via volumetric occlusion zones with density-dependent visibility, deriving a closed-form approximation for the probability that a marker is seen by at least two cameras [11]. In each case, the designer recognised that the arrangement of cameras,not merely their presence, determines whether the system can produce a usable measurement.

The most recent and arguably most honest formulation of the OCP treats it as an explicitly multi-objective problem, recognising that coverage and accuracy are not independent and that neither alone captures the full cost of a poor configuration. Ziegler et al. maximised the measurement volume whilst enforcing convergence angle constraints, combining a coverage objective with a geometric accuracy requirement in a single optimisation [7]. Zhou et al. constructed a multi-objective function that balanced reconstruction error against spatial coverage, weighted by designer-specified priorities [8]. Aissaoui et al. integrated a composite error metric at the GA evaluation level that combined field-of-view coverage, resolution degradation, and occlusion

into a single weighted fitness score [3]. Several authors further incorporated budget constraints, seeking either to minimise the number of cameras required to achieve a specified quality threshold [4, 11] or to maximise quality for a fixed camera count [2, 3]. The field has converged toward the recognition that single-objective formulations misrepresent the problem. The question is no longer whether to combine objectives but how to weight them, and this weighting inevitably reflects the priorities of the specific application. Table **??** summarises the classification of the reviewed literature across solver type, placement space, and objective category.

Depending on the application, available resources, and workspace constraints, different optimisation objectives have been prioritised in the literature. Broadly, these can be categorised into coverage-based objectives and accuracy-based objectives. In terms of coverage-based objectives, the goal is to maximise the visible area of the workspace, and sometimes going further to handle aspects of occlusion both static and dynamic and hence usually ensuring a two-cover coverage. This is particularly relevant in surveillance applications where ensuring that all regions are monitored is critical [6]. Conversely, accuracy-based objectives focus on minimising localisation error, which is crucial in motion capture and augmented reality applications where precise tracking is required and more than one camera arranged sufficiently non-parallel is needed to ensure triangulation is possible [2, 3]. Wu, Sharma, and Huang proposed another accuracy-based metric of placement which assessed how the limited camera intrinsics and quantisation lead to inaccurate measurements exacerbated by the placement of two cameras [1]. Many studies have proposed hybrid objectives that balance the trade-off of accuracy with coverage [7, 8]. Some works have also considered additional constraints such as budget limits and physical mounting options [3, 9, 11]. The configuration quality for 3D marker tracking and trinagulation is mostly affected by two main sources of error: marker visibility and triangulation accuracy. In terms of the application being MoCap, a marker should not be occluded by any obstacle, including itself, the marker must be within the cameras field of view, vissible given the camera's resolution and range of capture. These are necessary to ensure consistent and continual tracking. Furthermore, at least two cameras should be arranged sufficiently non-parallel so that triangulation is well-conditioned. Sometimes, the easy answer is just to throw more cameras at the problem as that does give redundant views and ensures we can place camera pairs to cover the scene in a sufficently non-parallel format but in most research situationships, it is not prudent to over-exhaust the budget and hence it may be a desirable goal to asses the least amount of cameras required to achieve a level of accuracy/ coverage to minimise cost.

5.1.4.2 Validation Approaches in the Literature

A persistent weakness of the OCP literature is the gap between simulated optimality and demonstrated real-world improvement. Most studies validate their optimisers exclusively in simulation, comparing the cost function value of the optimised configuration against that of a baseline (typically random or uniformly distributed cameras) without deploying the resulting configuration on physical hardware. Olague and Mohr evaluated their GA-optimised configurations by computing the covariance of

Table 5.1: Classification of the reviewed OCP literature by solver, placement space, and optimisation objective.

Source	Solver	Space	Objective	Key Metrics
Kirchhof (2013)	BIP / NLP	Discrete / Continuous	Accuracy	GDOP; min. sensors
Dai	BIP + Greedy	Discrete	Hybrid	Occlusion probability; min. cameras
Bodor et al. (2007)	Hill-climbing	Discrete (refined)	Coverage	Task-weighted observability
Olague & Mohr (2002)	GA (real-valued)	Continuous	Accuracy	Covariance of 3D reconstruction
Aissaoui et al. (2018)	Guided GA	Continuous	Hybrid	Reconstruction error; visibility; resolution; occlusion
Rahimian & Kearney (2017)	SA	Continuous	Accuracy	Dynamic occlusion + triangulability
Zhou et al. (2024)	Hybrid PSO + GA	Discrete	Hybrid	Coverage + reconstruction or

reconstructed 3D points in simulation and comparing against heuristic placements, but did not report physical experiments [5]. Kirchhof validated the GDOP-based metric against both his BIP and NLP solvers in simulation but acknowledged that experimental deployment was beyond the scope of the work [4]. Zhou et al. compared their hybrid PSO-GA solutions against individual PSO and GA runs, demonstrating superior convergence in simulation without physical validation [8].

The most substantive experimental validation in the reviewed literature is that of Rahimian and Kearney, who deployed their simulated-annealing-optimised camera configurations on a physical OptiTrack system and compared tracking completeness against an ad-hoc arrangement [2]. Their metric was the percentage of captured frames in which all 37 markers on a moving human subject were successfully reconstructed: the optimised placement achieved full marker reconstruction on 86% of frames, compared to only 30% under the ad-hoc configuration. This nearly three-fold improvement provides the most direct evidence in the literature that simulation-based OCP translates to measurable real-world gains. Aissaoui et al. similarly validated their guided GA by comparing the reconstruction error of a physical multi-camera system before and after optimisation, reporting reduced marker dropout rates in the optimised configuration [3]. Zhao et al. conducted real-life experiments with a seven-camera surveillance network to validate their BIP formulation against greedy and LP-relaxation solvers, measuring observability of walking subjects in a 7.6×3.7 m area [**zhaoApproximateTechniquesSolving2013**].

The question of what constitutes ground truth for validating a camera placement optimiser warrants careful consideration. In MoCap, ground truth refers to the true three-dimensional position of the tracked markers, against which the system's reconstructed positions are compared. Optical motion capture systems such as OptiTrack are themselves frequently used as the ground truth reference for validating other tracking modalities (inertial measurement units, visual odometry, LiDAR-based localisation), owing to their sub-millimetre precision when properly calibrated. Aurand et al. systematically mapped the accuracy of a 42-camera OptiTrack Prime 41 system across a 135 m^3 capture volume by comparing single-marker motion against a ThorLabs linear motion stage with 5 μm positioning accuracy, finding that 97% of the capture area exhibited errors below 200 μm [**aurandAccuracyMapOptical2017**]. Nagymáté and Kiss provided a comprehensive review of OptiTrack validation studies, noting that the majority of the biomechanical literature under-documents system setup parameters (camera type, number, frame rate, marker size), which undermines repeatability [**nagymate2018optitrack**]. This observation reinforces the importance of fully specifying the camera configuration, which is precisely what the OCP optimiser determines.

For the present work, the primary validation pathway is to compare the tracking performance of the optimised camera configuration against the default ad-hoc arrangement on the physical OptiTrack system, using metrics that bridge the gap between the simulation cost function and observable tracking quality. The calibration residual reported by OptiTrack's Motive software, which represents the mean offset between converging rays during 3D marker reconstruction, provides a direct measure

of the system's triangulation precision for a given camera arrangement. Additionally, marker reconstruction completeness (the fraction of frames in which all markers are successfully resolved) and spatial coverage uniformity can be assessed empirically. These metrics are discussed further in Section **??**.

5.1.5 Summary

The measurement uncertainty attributed to the spatial relationship between the target and the cameras can be modelled based on the distance between the target and camera which can excerbated pixel resolution and the orientation between camera's view vectors where the increasingly more parallel these vectors are results in ill-conditioned triangulation geometry, culminating in poor pose estimates and hence poor tracking

As seen there have been a plethora of proposed methodologies for optimising camera placement in the literature which are summarised in the timeline below where they have been broadly categorised as heuristic optimisation or binary integer programming and their objectives have also been listed based on thematic colour rings indicated in the key. .3

The optimal camera placement problem extends the AGP foundation by introducing constraints that reflect the true capabilities and limitations of physical sensors. These extensions transform camera placement from a geometric covering problem into a constrained optimisation problem where the cost function must encode both visibility and measurement quality

Generally, the OCP can be stated as follows: given a set of candidate camera locations and orientations, find the configuration that maximises 3D reconstruction quality of a target within a defined workspace, subject to practical constraints such as mounting options and budget. Solving this problem requires balancing trade-offs among coverage, accuracy, and computational efficiency. Without the aid of an optimisation algorithm, the sensors are arranged in an ad-hoc configuration by the designer, which requires expertise or trial and error, which can be time-consuming or sacrificial if a sub-par arrangement is settled on with no mathematical quantitative guidance. Thus, the resulting localisation error is neither calculated nor considered mathematically. Here lies the need to optimise the camera network based on metrics that minimise degrading effects like resolution uncertainty and probabilistic occlusion even if the trajectory of the target is not known exactly beforehand.

5.2 Theoretical Framework

5.2.1 Genetic Algorithm Terminology

The language used to describe genetic algorithms borrows heavily from evolutionary biology. Table **??** defines these terms in both domains to clarify the analogy and establish consistent vocabulary for the sections that follow.

This biological analogy provides useful intuition: just as natural selection refines

Table 5.2: Comparison of biological and genetic algorithm terminology.

Term	Biological Meaning	Genetic Algorithm Meaning
Chromosome	A structure of DNA carrying genetic information	A complete candidate solution encoded in some representation, e.g bit strings
Gene	A unit of heredity determining a specific trait	A single element or parameter within the candidate solution, e.g a bit
Population	A group of interbreeding individuals of the same species	The set of candidate solutions existing at a given iteration
Fitness	An organism's ability to survive and reproduce in its environment	An objective function quantifying how well a candidate solution achieves the desired goal
Generation	All individuals born and living at approximately the same time	One complete iteration of the algorithm producing a new population
Selection	Differential survival and reproduction due to heritable trait variation	An operator that chooses higher-performing individuals to serve as parents
Crossover	Exchange of genetic material between homologous chromosomes during meiosis	An operator that combines portions of two parent solutions to create offspring
Mutation	A random, heritable change in genetic sequence, ususally caused by a erroneous DNA replication	An operator that randomly perturbs one or more genes in a solution

a species over generations by favouring advantageous traits, genetic algorithms iteratively improve a population of solutions by favouring those with higher fitness. While genetic algorithms lack the biological complexity of true evolution, they do provide a productive conceptual framework. The effectiveness of this process depends on how solutions are represented, this is often reffered to as the encoding.

5.2.2 Encoding Schemes

"coming up with the best encoding is tantamount to solving the problem itself" difference between binary encoding, gray code, real encoding, variable length chromosomes

Binary encoding advantages: mutation and crossover are developed based on this type of encoding. limitations: Hamming cliff effect, encoding and decoding, computer based level of precision (binary precision)

Gray code: advantages: removes hamming cliff artefact limitations: not popularised

Real encoding advantages: no encoding and subsequent decoding step required, avoids hamming cliff artefact, more suited for a continous exploration limitations: mutation and crossover need to be restructured explain that this was chosen for implementation -> how mutation and crossover were altered

5.2.3 Selection Methods

Fitness proportionate selection: roulette-wheel selection, stochastic universal sampling number of times an individual is expected to reproduce is equal to its fitness divided by the average fitness in the population roulette wheel: each individual is assigned a slice of the circular roulette wheel

rank-based selection tournament selection boltzmann selection elitism

explain why tournament and elitism were chosen -> monotomic convergence (always improving or consistent best fitness)

5.2.4 Crossover Techniques

single-point crossover double point crossover parameterized uniform crossover

5.2.5 Convergence Analysis

iteration based fitness score based

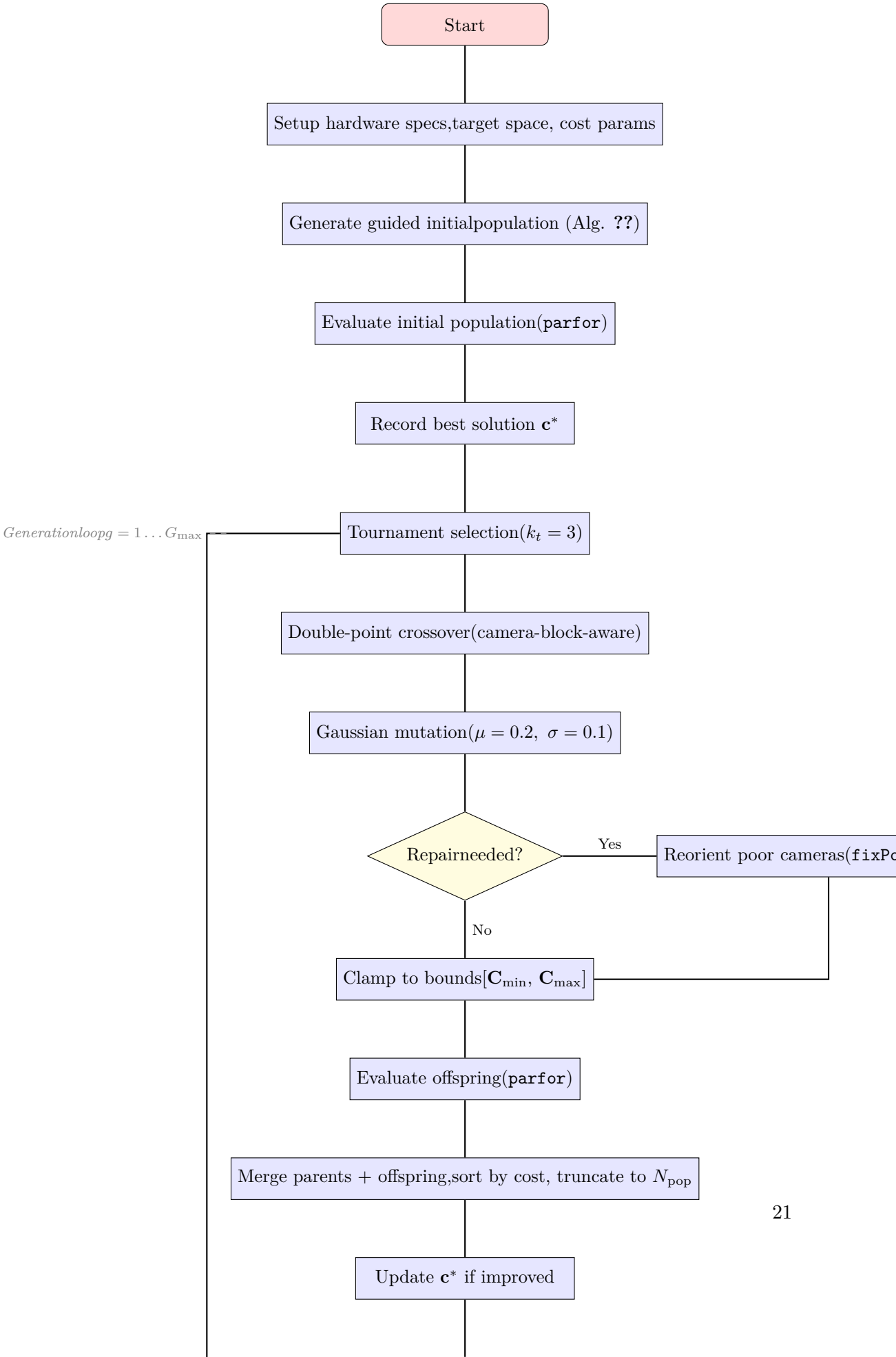
5.3 Methodology

5.4 Methodology

This section describes the implementation of the real-coded guided genetic algorithm developed to solve the OCP for a seven-camera OptiTrack motion capture system. The algorithm operates on a continuous placement space, directly encoding camera positions and orientations as real-valued chromosomes, and evaluates candidate configurations against a composite cost function that captures both resolution uncertainty and dynamic occlusion. The overall workflow is illustrated in Figure **??**.

5.4.1 Algorithm Configuration

A real-coded genetic algorithm was selected as the optimisation solver for three principal reasons. First, the camera placement variables, three spatial coordinates (x, y, z) and three Euler angles (α, β, γ) per camera, are inherently continuous. A binary encoding would require a discretisation step that introduces quantisation artefacts and constrains the solution to a finite grid, as discussed in Section **??**. Olague and Mohr demonstrated the suitability of real-valued chromosomes for camera placement by encoding continuous camera parameters directly, avoiding the Hamming cliff problem inherent to binary representations [5]. Aissaoui et al. adopted the same representation for their guided genetic algorithm applied to MoCap camera networks [3]. Second, the cost function employed here (Section


Start
Setup hardware specs,target space, cost params
Generate guided initialpopulation (Alg. ??)
Evaluate initial population(parfor)
Record best solution $\mathbf{c}^*$
$Generationloopg = 1 \ldots G_{\max}$
Tournament selection($k_t = 3$)
Double-point crossover(camera-block-aware)
Gaussian mutation($\mu = 0.2,\ \sigma = 0.1$)
Repairneeded?
Yes
Reorient poor cameras(fixPo
No
Clamp to bounds[$\mathbf{C}_{\min}$, $\mathbf{C}_{\max}$]
Evaluate offspring(parfor)
Merge parents + offspring,sort by cost, truncate to N_{pop}
Update $\mathbf{c}^*$ if improved

??) involves ray-casting visibility checks, covariance-based uncertainty estimation, and probabilistic occlusion modelling, none of which admit linearisation into the algebraic form required by BIP or MILP solvers. The GA treats the cost function as a black box, requiring only a scalar fitness value for each candidate, which permits arbitrarily complex objective functions. Third, the population-based nature of the GA provides broad exploration of the search space, mitigating the risk of entrapment in local minima that single-point solvers like gradient descent or simulated annealing are more susceptible to.

5.4.1.1 Chromosome Representation

Each candidate camera configuration is encoded as a single real-valued chromosome of length $6k$, where k is the number of cameras. For the seven-camera system considered here, $k = 7$ and the chromosome length is $n_{\text{var}} = 42$. The chromosome is structured as k contiguous six-gene blocks:

$$\mathbf{c} = [\underbrace{x_1, y_1, z_1, \alpha_1, \beta_1, \gamma_1}_{\text{Camera 1}}, \underbrace{x_2, y_2, z_2, \alpha_2, \beta_2, \gamma_2}_{\text{Camera 2}}, \ldots, \underbrace{x_k, y_k, z_k, \alpha_k, \beta_k, \gamma_k}_{\text{Camera } k}] \tag{5.1}$$

where (x_i, y_i, z_i) specifies the position of camera i in the world frame and $(\alpha_i, \beta_i, \gamma_i)$ are the Euler angles (XYZ convention) defining its orientation. This block structure is fundamental to the design of the crossover and mutation operators: the six genes constituting a single camera are treated as an atomic unit to preserve the physical coupling between a camera's position and its orientation. Splitting position from orientation during crossover would produce offspring where a camera points in a direction that was optimised for an entirely different position, yielding invalid configurations.

The OptiTrack system under consideration comprises a configurable mix of narrow-angle cameras (focal length $f = 5.5$ mm) and wide-angle cameras (focal length $f_w = 3.5$ mm). When wide-angle lenses are available, the lens assignment is deterministic by chromosome slot index rather than encoded as an additional gene: cameras at designated slot indices receive the wide-angle lens, and the remaining cameras receive the narrow-angle lens. For example, in the default seven-camera configuration, cameras at even-indexed slots are assigned wide-angle lenses, yielding three wide-angle and four narrow-angle cameras. This deterministic assignment ensures that crossover and mutation cannot violate the hardware constraint on the number of each lens type, since the lens type is a property of the slot, not of the genes occupying it. When all cameras share the same lens type, this assignment reduces to a uniform intrinsic configuration and the distinction is transparent to the GA operators. The corresponding effective detection ranges are 16 m for the narrow-angle and 9 m for the wide-angle lenses, as specified in the OptiTrack datasheet. This design is parameterised via a configurable list of wide-angle slot indices (`wideAngleCamIndices`), allowing the user to adapt the system to any available lens inventory without modifying the GA logic.

5.4.1.2 Search Space Bounds

The search space is defined by the physical workspace and mounting constraints. The workspace volume for the target space is $[-4, 4] \times [-4, 4] \times [0, 4]$ m, matching the available flight envelope. Camera positions are permitted within slightly enlarged bounds to allow wall and ceiling mounting:

$$\mathbf{c}_{\min} = [-5,\ -4.5,\ 0,\ -\pi,\ -\pi,\ -\pi], \qquad \mathbf{c}_{\max} = [5,\ 4.5,\ 4.8,\ \pi,\ \pi,\ \pi] \tag{5.2}$$

The position bounds extend beyond the target volume to model mounting on walls and the ceiling, while the orientation bounds span the full $[-\pi, \pi]$ range for each Euler angle. These per-camera bounds are tiled across all k cameras to produce the chromosome-level bounds $\mathbf{C}_{\min} = \text{repmat}(\mathbf{c}_{\min}, 1, k)$ and $\mathbf{C}_{\max} = \text{repmat}(\mathbf{c}_{\max}, 1, k)$. After mutation, any gene that exceeds its bound is clamped to the nearest boundary value.

5.4.1.3 Parameter Selection

The GA parameters governing the evolutionary search are summarised in Table **??**. These values were selected based on preliminary experimentation and conventions established in the OCP literature.

Table 5.3: Genetic algorithm parameter settings.

Parameter	**Symbol**	**Value**
Population size	N_{pop}	150
Maximum generations	$G_{\max}$	150
Crossover fraction	p_C	1.0
Mutation probability	μ	0.2
Mutation step size	σ	0.1
Tournament size	k_t	3

A population size of 150 balances diversity against computational expense. Each fitness evaluation requires projecting every target point through every camera and, for the combined cost function, computing both the resolution uncertainty and the occlusion metric. With parallel evaluation across six workers, a population of 150 completes a single generation in a reasonable time whilst maintaining sufficient genetic diversity to avoid premature convergence.

The crossover fraction $p_C = 1.0$ means that the entire population participates in crossover, producing N_{pop} offspring per generation. Combined with the elitist replacement strategy (Section **??**), this aggressive crossover rate ensures thorough exploration: all parents contribute genetic material, but only the best N_{pop} individuals from the combined parent-offspring pool survive to the next generation.

The mutation probability $\mu = 0.2$ specifies the per-gene probability of mutation. On average, $0.2 \times 42 \approx 8$ genes are perturbed per offspring, which introduces sufficient variation to escape local optima without being so disruptive as to destroy good solutions. The mutation step size $\sigma = 0.1$ controls the standard deviation of the

Gaussian perturbation applied to mutated genes. Since position genes are measured in metres and orientation genes in radians, this value produces perturbations of approximately ± 0.2 m and ± 0.2 rad at one standard deviation, which are physically meaningful adjustments for the workspace dimensions considered.

5.4.1.4 Initial Population

An unguided random initialisation distributes cameras uniformly throughout the search volume, which would place many cameras in the interior of the workspace where they cannot observe targets, or orient them away from the target space. Aissaoui et al. demonstrated that seeding the initial population with domain knowledge significantly accelerates convergence by restricting the search to physically plausible configurations from the outset [3]. The implementation adopts a guided initialisation strategy that places cameras on the boundary faces of the workspace and orients them toward the interior.

The workspace is first subdivided into $\lfloor k/2 \rfloor$ sections, where k is the number of cameras. The section centres are distributed on a balanced two-dimensional grid at the mid-height of the workspace. If the number of sections is odd, one section centre is placed at the geometric centre of the volume, with the remaining centres distributed on the grid. These section centres serve as initial aiming targets for the cameras.

For each camera in the initial chromosome, a position is generated by randomly selecting one of five boundary faces (four walls and the ceiling, excluding the floor) and sampling uniformly on that face. The camera is then oriented toward the nearest section centre by computing the direction vector from the camera position to the centre and converting it to Euler angles via an axis-angle to quaternion to Euler angle decomposition. This procedure is summarised in Algorithm **??**.

Algorithm 1: Guided Initial Population Generation

Bounds $\mathbf{c}_{\min}$, $\mathbf{c}_{\max}$; section centres $\mathbf{S}$; number of cameras k Chromosome $\mathbf{c}$ of length $6k$ $i = 1$ k Randomly select boundary face $f \in \{+X, -X, +Y, -Y, +Z\}$ Sample position $\mathbf{p}_i$ uniformly on face f Find nearest section centre: $\mathbf{s}^* = \arg\min_{\mathbf{s}\in\mathbf{S}} \|\mathbf{s} - \mathbf{p}_i\|$ Compute direction: $\hat{\mathbf{d}} = (\mathbf{s}^* - \mathbf{p}_i)/\|\mathbf{s}^* - \mathbf{p}_i\|$ Compute rotation from $\hat{\mathbf{z}}$ to $\hat{\mathbf{d}}$ via axis-angle: $\theta = \arccos(\hat{\mathbf{z}} \cdot \hat{\mathbf{d}})$, $\hat{\mathbf{k}} = \hat{\mathbf{z}} \times \hat{\mathbf{d}}$ Convert to quaternion: $\mathbf{q} = [\cos(\theta/2),\ \hat{\mathbf{k}}\sin(\theta/2)]$ Extract Euler angles: $(\alpha_i, \beta_i, \gamma_i) \leftarrow \text{quat2eul}(\mathbf{q})$ $\mathbf{c}[(i-1)\cdot 6 + 1 : i \cdot 6] \leftarrow [\mathbf{p}_i,\ \alpha_i,\ \beta_i,\ \gamma_i]$ $\mathbf{c}$

By construction, every camera in the initial population is positioned on the perimeter of the workspace and aimed inward. This eliminates the generation-wasting phase in which an unguided GA must first evolve cameras from arbitrary interior positions to useful boundary positions. The floor face is excluded because a floor-mounted camera looking upward would observe only the ceiling, contributing negligibly to tracking coverage in the target volume.

5.4.1.5 Tournament Selection

Selection determines which individuals from the current population are chosen as parents for crossover. The implementation uses tournament selection with a tournament size of $k_t = 3$. In each tournament, k_t individuals are drawn uniformly at random (with replacement) from the population, and the individual with the lowest cost (highest fitness) is selected as a parent. This process is repeated N_C times to fill the parent pool, where $N_C = p_C \cdot N_{\text{pop}}$ is the number of offspring to be produced.

Tournament selection was chosen over roulette wheel (fitness-proportionate) selection for two reasons. First, roulette wheel selection is sensitive to the scaling of the fitness function: when the population converges and fitness values cluster, the selection pressure diminishes because all individuals receive nearly equal selection probabilities. Conversely, when one individual dominates, it consumes a disproportionate share of the probability mass, reducing diversity. Tournament selection avoids both failure modes because it depends only on the rank ordering of the k_t sampled individuals, not on the absolute or relative magnitudes of their fitness values. Second, the tournament size k_t provides a direct and intuitive control over selection pressure. A tournament size of $k_t = 2$ provides mild pressure (the better of two random individuals), while larger values increase the probability that the population's best individuals are selected. A value of $k_t = 3$ was chosen as a standard compromise that maintains diversity whilst still preferring high-quality parents.

The selection is implemented in a vectorised manner for computational efficiency. All N_C tournaments are constructed simultaneously by drawing an $N_C \times k_t$ matrix of random population indices. The costs of all participants are retrieved via a single indexing operation, and the column-wise minimum identifies the winner of each tournament, avoiding an explicit loop over individual tournaments.

5.4.1.6 Double-Point Crossover

Crossover recombines genetic material from two parent chromosomes to produce two offspring. The implementation uses double-point crossover operating on complete camera blocks rather than individual genes. Two crossover points j_1 and j_2 (where $j_1 < j_2$) are selected uniformly at random from the set $\{1, 2, \ldots, k\}$ of camera indices. The crossover points are then converted to gene indices: $g_1 = 6j_1$ and $g_2 = 6j_2$. The offspring are formed by exchanging the camera blocks between the two crossover points:

$$\mathbf{y}_1 = [\mathbf{x}_1(1 : g_1), \quad \mathbf{x}_2(g_1{+}1 : g_2), \quad \mathbf{x}_1(g_2{+}1 : \text{end})] \tag{5.3}$$

$$\mathbf{y}_2 = [\mathbf{x}_2(1 : g_1), \quad \mathbf{x}_1(g_1{+}1 : g_2), \quad \mathbf{x}_2(g_2{+}1 : \text{end})] \tag{5.4}$$

where $\mathbf{x}_1$ and $\mathbf{x}_2$ are the parent chromosomes. This block-aware crossover ensures that a camera's position and orientation are always inherited together. If crossover were permitted to split within a six-gene camera block, an offspring could inherit a position from one parent and an orientation from the other. Since each parent's

orientation was optimised for its corresponding position, such a recombination would produce a camera pointing in a direction that is geometrically inconsistent with its location, equivalent to injecting noise rather than recombining useful building blocks.

Double-point crossover was preferred over single-point crossover because single-point crossover exhibits positional bias: genes near the ends of the chromosome are less likely to be separated than genes near the centre. With two crossover points, any contiguous block of cameras can be exchanged with equal probability, providing a more uniform exploration of the combinatorial space of camera subsets. Because the lens type is determined by the slot index and not by the genes, crossover cannot violate the wide-angle and narrow-angle camera count constraint regardless of which blocks are exchanged.

5.4.1.7 Mutation

Mutation introduces random perturbations to offspring chromosomes to maintain genetic diversity and prevent premature convergence. Each gene in the offspring chromosome is independently mutated with probability μ. A mutated gene receives a Gaussian perturbation:

$$c'_j = c_j + \sigma \cdot \mathcal{N}(0,1) \qquad \text{if rand}() < \mu \tag{5.5}$$

where c_j is the original gene value, σ is the mutation step size, and $\mathcal{N}(0,1)$ is a standard normal random variable. After mutation, all genes are clamped to their respective bounds: $c'_j \leftarrow \max(c'_j, C_{\min,j})$ and $c'_j \leftarrow \min(c'_j, C_{\max,j})$.

The Gaussian mutation operator is the natural choice for real-coded chromosomes because it produces small perturbations with high probability and large perturbations with low probability, enabling fine-grained local search most of the time whilst retaining the ability to escape local optima through occasional large jumps. A uniform mutation operator, by contrast, would produce perturbations of arbitrary magnitude with equal probability, which is overly disruptive for a search space where near-optimal solutions often differ from each other by small positional or angular adjustments.

Camera Repair Operator. In addition to standard Gaussian mutation, a repair operator is applied periodically to correct cameras that contribute minimally to the network's coverage. The repair operator, referred to as `fixPoorCameras`, checks whether each camera in the offspring can project at least a threshold fraction (5%) of the target points onto its image plane. Cameras falling below this threshold are assumed to be oriented away from the target space and are reoriented toward the nearest section centre, using the same axis-angle procedure as the initial population generator (Section **??**). The camera's position is left unchanged; only its three orientation genes are overwritten. This repair is applied every 10th generation deterministically, and with a 20% probability in each other generation. This mixed schedule avoids the computational expense of repairing every offspring in every generation whilst ensuring that grossly misconfigured cameras are corrected before

they consume evaluation budget. The repair operator represents a form of Lamarckian learning within the evolutionary framework: knowledge about the problem structure (cameras should face the workspace) is used to correct phenotypically deficient individuals, a strategy analogous to Aissaoui et al.'s guided initialisation but applied throughout the search rather than only at the start [3].

5.4.1.8 Elitism

Elitism ensures that the best solution found so far is never lost due to the stochastic nature of selection, crossover, and mutation. The implementation uses a $(\mu + \lambda)$ replacement strategy: the parent population of size N_{pop} is merged with the offspring population of size N_C, the combined pool is sorted by cost in ascending order, and the best N_{pop} individuals are retained as the next generation. This is equivalent to deterministic truncation selection applied to the union of parents and offspring.

This strategy guarantees monotonic convergence: the best cost in the population can never increase from one generation to the next, because the best individual from the previous generation is always present in the merged pool and will survive truncation unless displaced by a superior offspring. Monotonic convergence is a valuable diagnostic property; any observed increase in the best cost across generations would indicate a software defect, such as updating the best solution from the wrong population array. The $(\mu + \lambda)$ strategy also provides implicit elitism without requiring a separate elite archive or explicit best-individual preservation, simplifying the implementation whilst achieving the same effect.

The complete GA loop is summarised in Algorithm **??**.

Algorithm 2: Real-Coded Guided Genetic Algorithm for Camera Placement

Cost function f; bounds $\mathbf{C}_{\min}$, $\mathbf{C}_{\max}$; parameters N_{pop}, $G_{\max}$, μ, σ, k_t, p_C
Best camera configuration $\mathbf{c}^*$ and cost f^* Initialise population
$P = \{\mathbf{c}_1, \ldots, \mathbf{c}_{N_{\text{pop}}}\}$ using Algorithm **??** Evaluate $f(\mathbf{c}_i)$ for all $i \in \{1, \ldots, N_{\text{pop}}\}$
(parallel) $\mathbf{c}^* \leftarrow \arg\min_{\mathbf{c} \in P} f(\mathbf{c})$; $f^* \leftarrow f(\mathbf{c}^*)$ $g = 1$ $G_{\max}$ Select N_C parents
via tournament selection (size k_t) from P $j = 1$ $N_C/2$
$(\mathbf{y}_{2j-1}, \mathbf{y}_{2j}) \leftarrow \text{DoublePointCrossover}(\mathbf{x}_{2j-1}, \mathbf{x}_{2j})$ *(camera-block-aware)* $j = 1$
N_C $\mathbf{y}_j \leftarrow \text{Mutate}(\mathbf{y}_j, \mu, \sigma)$ $g \bmod 10 = 0$ $\text{rand}() < 0.2$
$\mathbf{y}_j \leftarrow \text{fixPoorCameras}(\mathbf{y}_j)$ *(repair operator)* $\mathbf{y}_j \leftarrow \text{clamp}(\mathbf{y}_j, \mathbf{C}_{\min}, \mathbf{C}_{\max})$
Evaluate $f(\mathbf{y}_j)$ for all $j \in \{1, \ldots, N_C\}$ *(parallel)* $\min_j f(\mathbf{y}_j) < f^*$
$\mathbf{c}^* \leftarrow \arg\min_j \mathbf{y}_j$; $f^* \leftarrow \min_j f(\mathbf{y}_j)$ $P \leftarrow \text{sort}(P \cup \{\mathbf{y}_1, \ldots, \mathbf{y}_{N_C}\})$ by
ascending cost $P \leftarrow P(1 : N_{\text{pop}})$ *(truncation / elitism)* $\mathbf{c}^*$, f^*

5.4.2 Cost Function

The cost function quantifies the quality of a candidate camera configuration by measuring two complementary sources of localisation error: resolution uncertainty arising from pixel quantisation, and dynamic occlusion arising from self-occlusion of the target. Each cost component is evaluated over a discrete grid of target points

sampling the workspace volume, and the results are combined into a single scalar cost via weighted normalisation.

The target space is generated as a three-dimensional grid of points within the workspace volume. Two discretisation modalities are supported: a uniform grid with constant spacing (suitable for full-volume ?? tracking), and a normally-concentrated grid generated via inverse CDF warping that concentrates points toward the centre of the volume (capturing the higher probability of target presence near the workspace centre). The grid spacing Δ controls the trade-off between evaluation fidelity and computational cost; a spacing of 1 m produces a manageable number of evaluation points whilst capturing the spatial variation of both cost components.

5.4.2.1 Resolution Uncertainty Cost Function

The resolution uncertainty cost function quantifies the positional ambiguity that arises when a three-dimensional target point is projected onto a discrete pixel grid. This formulation is based on the analytical uncertainty bounds derived by Wu, Sharma, and Huang [1], who showed that the volume of the three-dimensional uncertainty region at a target point depends on the camera-target geometry, the pixel quantisation step, and the number of observing cameras.

For a given target point $\mathbf{p}$, each camera i projects $\mathbf{p}$ onto its image plane to obtain pixel coordinates (u_i, v_i). If these coordinates lie within the image bounds ($1 \leq u_i \leq W$ and $1 \leq v_i \leq H$, where $W \times H$ is the sensor resolution), the camera is considered to have visibility of the point. For each visible camera, the four corners of the pixel's quantisation cell are back-projected into three-dimensional space:

$$\mathbf{w}_{i,c} = \mathbf{R}_i \left(\mathbf{K}_i^{-1} \begin{bmatrix} u_i \pm \delta_u \\ v_i \pm \delta_v \\ 1 \end{bmatrix} \right) + \mathbf{t}_i, \qquad c \in \{1, 2, 3, 4\} \tag{5.6}$$

where $\mathbf{K}_i$ is the camera intrinsic matrix, $\mathbf{R}_i$ is the rotation matrix, $\mathbf{t}_i$ is the camera centre, and $\delta_u = \delta_v = 0.5$ pixels is half the quantisation step. These four back-projected rays, together with the camera centre, define a four-sided pyramid in three-dimensional space. The pyramid represents the set of all world points that could produce the observed pixel coordinates (u_i, v_i) within the quantisation uncertainty.

When multiple cameras observe the same target point, the intersection of their respective pyramids defines the three-dimensional uncertainty region, the volume of space within which the true target position is indistinguishable given the pixel-level measurements. The vertices of this polyhedral intersection are computed by finding the intersections of the bounding planes of each pyramid pair. Each pyramid contributes four bounding planes defined by adjacent corner pairs; for n_v visible cameras, $\binom{n_v}{2}$ camera pairs are considered, and for each pair, the three-plane intersections of adjacent surfaces from each pyramid are tested. A candidate vertex is retained only if it lies inside all visible pyramids (tested via the signed distance to each bounding plane).

The uncertainty at each target point is then computed from the covariance of the intersection vertices. The unique vertices are centred about the target point to form a zero-mean point cloud $\mathbf{V}_{\text{centred}}$, and the sample covariance matrix is computed:

$$\mathbf{C} = \text{cov}(\mathbf{V}_{\text{centred}}) \tag{5.7}$$

The eigenvalues $\lambda_1, \lambda_2, \lambda_3$ of $\mathbf{C}$ represent the variances along the principal axes of the uncertainty ellipsoid. The scalar uncertainty measure is the sum of the square roots of the eigenvalues:

$$U(\mathbf{p}) = \sum_{i=1}^{3} \sqrt{|\lambda_i|} \tag{5.8}$$

This metric is proportional to the sum of the semi-axis lengths of the uncertainty ellipsoid and captures the overall spread of the uncertainty region without requiring an explicit ellipsoid fit. An earlier implementation computed the uncertainty as the volume of a fitted ellipsoid ($\frac{4}{3}\pi abc$) using iterative optimisation (`fminsearch`), but the eigenvalue-based metric was adopted because it is significantly faster to compute, avoids the sensitivity of iterative methods to initial conditions, and yields comparable results when scaled appropriately.

Penalty values are assigned to target points with insufficient camera coverage: a full penalty $U_{\text{penalty}} = 100$ for points visible to zero cameras, and half the penalty for points visible to exactly one camera (which cannot be triangulated but are at least detected). The resolution uncertainty cost for the entire configuration is the mean uncertainty across all target points:

$$J_{\text{res}} = \frac{1}{N_p} \sum_{p=1}^{N_p} U(\mathbf{p}_p) \tag{5.9}$$

5.4.2.2 Dynamic Occlusion Cost Function

The dynamic occlusion cost function models the degradation in tracking quality caused by self-occlusion of the target by a vertical planar occluder. This formulation follows the probabilistic model of Rahimian and Kearney [2], which considers all possible orientations of the occluder and penalises configurations where the target cannot be triangulated from any direction.

For each target point $\mathbf{p}$, the cameras that have the point within their field of view and within their effective detection range are identified as visible. The effective range depends on the lens type: 16 m for narrow-angle cameras and 9 m for wide-angle cameras. For each visible camera i, the view vector $\hat{\mathbf{v}}_i = (\mathbf{p} - \mathbf{t}_i)/\|\mathbf{p} - \mathbf{t}_i\|$ is computed, representing the unit direction from the camera centre to the target. These view vectors are then projected onto the horizontal plane and their azimuthal angles $\phi_i = \text{atan2}(\hat{v}_{i,y}, \hat{v}_{i,x})$ are computed and sorted.

The sorted azimuthal angles partition the 360° around the target into angular sections. For each section, the cameras on the front side of a hypothetical vertical occluder oriented at the section's midpoint are identified. A camera is on the

front side if its azimuthal angle differs from the test angle by more than 90° and less than 270°. Among the front cameras, all pairwise combinations are tested for triangulability: two cameras can triangulate the target if their three-dimensional convergence angle $\theta_{ij} = \arccos(\hat{\mathbf{v}}_i \cdot \hat{\mathbf{v}}_j)$ satisfies:

$$\theta_{\min} \leq \theta_{ij} \leq \theta_{\max} \tag{5.10}$$

where $\theta_{\min} = 40°$ and $\theta_{\max} = 140°$ are the minimum and maximum convergence angles for well-conditioned triangulation. If no pair of front cameras satisfies this constraint, the section is classified as occluded and its angular width is added to the occlusion error. The total occlusion error for a target point is the sum of occluded section angles:

$$Q(\mathbf{p}) = \sum_{s \in \text{sections}} \omega_s \cdot \mathbb{1}[\text{not triangulable from front}] \tag{5.11}$$

where ω_s is the angular width of section s in degrees. A target visible to fewer than two cameras receives the maximum penalty of 360° (and an additional 360° if visible to zero cameras). The dynamic occlusion cost for the entire configuration is the mean occlusion angle:

$$J_{\text{occ}} = \frac{1}{N_p} \sum_{p=1}^{N_p} Q(\mathbf{p}_p) \tag{5.12}$$

5.4.2.3 Combined Cost Function

The combined cost function integrates both error sources into a single scalar objective using a weighted sum with fixed normalisation constants:

$$J = w_{\text{res}} \cdot \frac{J_{\text{res}}}{J_{\text{res}}^{\text{norm}}} + w_{\text{occ}} \cdot \frac{J_{\text{occ}}}{J_{\text{occ}}^{\text{norm}}} \tag{5.13}$$

where w_{res} and w_{occ} are the designer-specified weights (summing to 1), and $J_{\text{res}}^{\text{norm}}$ and $J_{\text{occ}}^{\text{norm}}$ are normalisation constants that bring both cost components to comparable scales. Without normalisation, the occlusion cost (measured in degrees, with raw values typically in the range 200–440) would dominate the uncertainty cost (with raw values typically in the range 20–100) by a factor of approximately 4–5×, causing the optimiser to effectively ignore resolution uncertainty.

The normalisation constants are set to fixed, round-number values: $J_{\text{res}}^{\text{norm}} = 100$ and $J_{\text{occ}}^{\text{norm}} = 440$, corresponding approximately to the cost of a random (unoptimised) camera configuration. Fixed constants were deliberately chosen over adaptive normalisation (e.g., normalising by the current population's mean or worst cost) because adaptive methods introduce a non-stationary fitness landscape. A configuration that has cost 0.5 in generation 10 might have cost 0.7 in generation 50 if the normalisation denominator changes, which can confuse the selection mechanism and violate the monotonic convergence guarantee of the elitist strategy. Fixed normalisation ensures that the cost function is deterministic for a given chromosome, which is essential for reproducible comparisons across runs.

The combined cost function thus permits the designer to control the trade-off between measurement precision and occlusion robustness via the weights w_{res} and w_{occ}. Equal weights ($w_{\text{res}} = w_{\text{occ}} = 0.5$) treat both objectives as equally important, while asymmetric weights can prioritise one source of error over the other depending on the application requirements.

5.4.3 Termination Criteria

The algorithm terminates after a fixed number of generations $G_{\max}$. An iteration-based termination criterion was selected for its simplicity and reproducibility: every run executes the same number of fitness evaluations, making runtime and computational budgets directly comparable across different configurations and parameter settings. The total number of cost function evaluations per run is $N_{\text{pop}} + G_{\max} \cdot N_C$, where $N_C = p_C \cdot N_{\text{pop}}$ is the number of offspring per generation. For the default parameters ($N_{\text{pop}} = 150$, $G_{\max} = 150$, $p_C = 1.0$), this amounts to $150+150\times150 = 22{,}650$ evaluations.

Alternative termination criteria based on convergence detection, such as terminating when the best cost has not improved by more than ϵ over the last n generations, were considered but not adopted. Such criteria risk premature termination if the algorithm encounters a plateau followed by a subsequent improvement, and they introduce additional hyperparameters (ϵ and n) that require tuning. The convergence behaviour is instead monitored post-hoc through the best-cost, population-average, and top-ten-average cost trajectories recorded at each generation (see Section **??**), allowing the adequacy of the generation budget to be assessed without influencing the search itself.

All cost function evaluations, both for the initial population and for the offspring in each generation, are parallelised across six workers using MATLAB's Parallel Computing Toolbox. Each target point's uncertainty or occlusion contribution is computed independently, enabling the inner evaluation loops to be distributed using `parfor`. This parallelisation reduces the wall-clock time per generation roughly proportional to the number of available workers, making the 150-generation budget feasible within practical time constraints.

5.5 Experimental Verification Framework

The optimisation algorithm developed in the preceding sections operates on a computational model of the camera network: projections are computed using an ideal pinhole camera model, visibility is determined by field-of-view bounds and range constraints, and the cost function aggregates resolution uncertainty and dynamic occlusion over a discrete grid of target points. While this model captures the dominant geometric factors that govern tracking quality, it necessarily omits secondary effects present in a physical deployment, including lens distortion, infrared illumination falloff, marker reflectivity variation with viewing angle, and environmental reflections. Consequently, the simulation-predicted cost of a configuration provides a necessary but not sufficient indicator of its real-world performance. This section describes the

framework for bridging the gap between simulated optimality and measured tracking quality.

5.5.1 Verification Objectives

The experimental verification has three objectives. First, to confirm that the optimised camera configuration produces measurably better tracking performance than the default ad-hoc arrangement on the physical OptiTrack system, thereby validating the premise that systematic placement optimisation yields tangible improvements. Second, to quantify the degree to which the simulation cost function correlates with observable tracking metrics, establishing whether the cost function is a reliable proxy for real-world performance. Third, to identify any discrepancies between the simulated and measured performance that would indicate model limitations or calibration sensitivities not captured by the optimiser.

5.5.2 Ground Truth and Reference Metrics

Establishing ground truth for a motion capture system presents a circularity: the system being evaluated is itself a high-precision measurement instrument. Two complementary strategies address this.

5.5.2.1 Internal Consistency Metrics

OptiTrack's Motive software reports a calibration residual after the wanding procedure, defined as the mean ray-convergence offset (in millimetres) across all calibration samples. This residual reflects the combined effect of camera placement geometry, lens distortion correction, and calibration sample quality. For a well-calibrated small-to-medium volume, residuals below 0.5 mm are typical; values below 0.3 mm indicate an excellent calibration. Comparing the calibration residual between the optimised and ad-hoc configurations, using an identical wanding protocol and the same capture volume, provides a direct measure of how placement geometry affects triangulation precision.

Additional internal metrics include:

- **Marker reconstruction completeness**: the percentage of captured frames in which all markers in the rigid body or skeleton are successfully reconstructed. This metric directly reflects the coverage and redundancy of the camera network, analogous to Rahimian and Kearney's primary validation metric [2].
- **Marker dropout rate**: the frequency with which individual markers are lost and regained during a capture sequence, indicating regions of marginal coverage or occlusion.
- **Spatial coverage uniformity**: the variance in reconstruction quality (residual or jitter) as a function of position within the capture volume, assessed by moving a rigid body through a structured grid of positions.

5.5.2.2 External Reference Measurements

For absolute accuracy assessment, a known geometric reference can be introduced into the capture volume. A rigid body with precisely measured inter-marker distances (known to within the manufacturing tolerance of the calibration fixture) serves as a portable ground truth. By placing this reference at multiple positions within the capture volume and comparing the OptiTrack-reported inter-marker distances against the known values, the absolute accuracy of the system can be mapped spatially. This approach follows the methodology of Aurand et al., who used a linear motion stage as an external reference to map accuracy across a large volume [**aurandAccuracyMapOptical2017**], but substitutes a rigid-body fixture that is more practical for the workspace dimensions considered here.

5.5.3 Experimental Protocol

The verification protocol consists of three phases, each conducted first with the ad-hoc (baseline) camera arrangement and then repeated with the optimised arrangement. The cameras, workspace, and lighting conditions remain identical between the two configurations; only the camera positions and orientations are changed.

1. **Calibration comparison**: Both configurations are calibrated using the same wanding protocol (identical wand type, comparable wanding duration and coverage). The calibration residual, overall and per-camera, is recorded. This phase isolates the effect of camera geometry on calibration quality.

2. **Static accuracy mapping**: A rigid body is placed at a structured grid of positions spanning the capture volume. At each position, a short static capture (several seconds) is recorded and the mean reconstructed inter-marker distances are compared against the known reference values. The resulting accuracy map reveals spatial variation in reconstruction precision for each configuration.

3. **Dynamic tracking assessment**: A representative motion sequence (e.g., a rigid body traversing a known path, or a human subject performing a standard movement protocol) is captured under both configurations. Marker reconstruction completeness, dropout rates, and frame-by-frame residuals are compared. For the dynamic case, the motion should exercise the full capture volume, including regions near the boundaries where coverage differences between configurations are expected to be most pronounced.

5.5.4 Linking Simulation to Measurement

The verification framework is designed to close the loop between the simulation cost function and physical performance. The resolution uncertainty cost component (Section **??**) predicts the spatial distribution of triangulation precision; the static accuracy map tests this prediction directly. The dynamic occlusion cost component (Section **??**) predicts vulnerability to marker dropout under self-occlusion; the dynamic tracking assessment tests this prediction by measuring reconstruction completeness during motion that induces occlusion events.

If the optimised configuration outperforms the ad-hoc baseline on the physical metrics, and the spatial pattern of improvement is consistent with the simulation predictions (e.g., the regions of largest simulated cost reduction coincide with the regions of largest measured accuracy improvement), then the cost function is validated as a meaningful proxy for real-world performance. Conversely, systematic discrepancies between simulation and measurement would identify directions for model refinement, such as incorporating lens distortion, illumination models, or environment-specific reflection sources into future iterations of the cost function.

5.6 Results and Discussion

5.6.1 Convergence Behaviour

5.6.2 Cost Function Evaluation

5.6.3 Parameter Sensitivity Analysis

5.6.4 Optimal Placement Configurations

Different Modalities Random Placement, ad hoc placement

5.6.5 System Limitations

? future work: if trajectory is known beforehand optimise the placement to be able to best recover the features/ tags by creating clusters of where the expected markers shall be like [13].

Conclusion and Future Work 6

6.1 Summary of Contributions

6.2 Key Findings

6.3 Limitations

includes scope constraints outlined in intro

6.4 Recommendations for Future Work

closed-loop robotic control Benchmark GA against other metaheuristic like PSO or SA to see if better solutions existan assess convergence performance

References

[1] R. Sharma, 'Analysis of uncertainty bounds due to quantization for three-dimensional position estimation using multiple cameras,' *Optical Engineering*, vol. 37, no. 1, p. 280, Jan. 1998, ISSN: 0091-3286. DOI: 10.1117/1.601615. Accessed: 22nd Aug. 2025.

[2] P. Rahimian and J. K. Kearney, 'Optimal Camera Placement for Motion Capture Systems,' *IEEE Transactions on Visualization and Computer Graphics*, vol. 23, no. 3, pp. 1209–1221, Mar. 2017, ISSN: 1941-0506. DOI: 10.1109/TVCG.2016.2637334. Accessed: 30th Jun. 2025.

[3] A. Aissaoui, A. Ouafi, P. Pudlo, C. Gillet, Z.-E. Baarir and A. Taleb-Ahmed, 'Designing a camera placement assistance system for human motion capture based on a guided genetic algorithm,' *Virtual Reality*, vol. 22, no. 1, pp. 13–23, Mar. 2018, ISSN: 1434-9957. DOI: 10.1007/s10055-017-0310-7. Accessed: 9th Jul. 2025.

[4] N. Kirchhof, 'Optimal placement of multiple sensors for localization applications,' in *International Conference on Indoor Positioning and Indoor Navigation*, Oct. 2013, pp. 1–10. DOI: 10.1109/IPIN.2013.6817862. Accessed: 14th Aug. 2025.

[5] G. Olague and R. Mohr, 'Optimal camera placement for accurate reconstruction,' *Pattern Recognition*, vol. 35, no. 4, pp. 927–944, Apr. 2002, ISSN: 0031-3203. DOI: 10.1016/s0031-3203(01)00076-0. Accessed: 9th Jul. 2025.

[6] R. Bodor, A. Drenner, P. Schrater and N. Papanikolopoulos, 'Optimal Camera Placement for Automated Surveillance Tasks,' *Journal of Intelligent and Robotic Systems*, vol. 50, no. 3, pp. 257–295, Oct. 2007, ISSN: 0921-0296, 1573-0409. DOI: 10.1007/s10846-007-9164-7. Accessed: 12th Aug. 2025.

[7] J. Ziegler, H. Gattringer and A. Müller, 'Optimal Camera Placement for Maximized Motion Capture Volume and Marker Visibility,' in *Advances in Service and Industrial Robotics*, T. Petrič, A. Ude and L. Žlajpah, Eds., Cham: Springer Nature Switzerland, 2023, pp. 45–52, ISBN: 978-3-031-32606-6. DOI: 10.1007/978-3-031-32606-6_6.

[8] J. Zhou, H. Deng, Z. Zhao, Y. Zou and X. Wang, *Sensor Placement Optimization of Visual Sensor Networks for Target Tracking Based on Multi-Objective Constraints*, https://www.mdpi.com/2076-3417/14/5/1722, Feb. 2024. Accessed: 14th Aug. 2025.

[9] V. A. Puligandla and S. Lončarić, 'A Multiresolution Approach for Large Real-World Camera Placement Optimization Problems,' *IEEE Access*, vol. 10, pp. 61 601–61 616, 2022, ISSN: 2169-3536. DOI: 10.1109/ACCESS.2022.3176817. Accessed: 14th Aug. 2025.

[10] X. Zhang, B. Zhang, X. Chen and Y. Fang, 'Coverage optimization of visual sensor networks for observing 3-D objects: Survey and comparison,' *International Journal of Intelligent Robotics and Applications*, vol. 3, no. 4, pp. 342–361, Dec. 2019, ISSN: 2366-598X. DOI: 10.1007/s41315-019-00102-6. Accessed: 14th Aug. 2025.

[11] X. Dai, 'Optimal Camera Configuration for Large-Scale Motion Capture Systems,'

[12] S. Jun, T.-W. Chang and H.-J. Yoon, 'Placing Visual Sensors Using Heuristic Algorithms for Bridge Surveillance,' *Applied Sciences*, vol. 8, no. 1, p. 70, Jan. 2018, ISSN: 2076-3417. DOI: 10.3390/app8010070. Accessed: 14th Aug. 2025.

[13] A. Aissaoui et al., 'Optimisation technique of camera placement for motion capture simulations,' *Computer Methods in Biomechanics and Biomedical Engineering*, vol. 17, no. sup1, pp. 122–123, Aug. 2014, ISSN: 1025-5842, 1476-8259. DOI: 10.1080/10255842.2014.931517. Accessed: 30th Jun. 2025.